
Set 1

MPI and OpenMP 2024-2025

Software & Programming of HPC Systems
(CEID_NE5407)

Στοιχεία Φοιτητών

Υπεύθυνος Καθηγητής:

Παναγιώτης Χατζηδούκας, Ευάγγελος Δερματάς, Ευστράτιος Γαλλόπουλος

Μέλος 1

Ακαδημαϊκό Έτος : 9^ο Εξάμηνο 2023-2024

Ονοματεπώνυμο: Ορέστης Αντώνιος Μακρής **A-M** 1084516

Μέλος 2

Ακαδημαϊκό Έτος : 9^ο Εξάμηνο 2023-2024

Ονοματεπώνυμο: Γρηγόρης Δελημπαλταδάκης **A-M** 1084647

Πανεπιστήμιο Πατρών Τμήμα Μηχανικών Η/Υ και Πληροφορικής

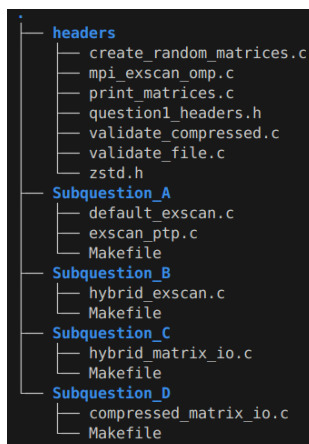
Πάτρα 2025

Contents

Question 1: Hybrid Programming Model and Parallel I/O	3
Επισκόπηση Δομής για την Ερώτηση 1	3
a) MPI Exclusive Scan Point-to-Point Υλοποίηση.....	3
Εξήγηση Κώδικα.....	3
Παράδειγμα Εξόδου	4
b) Hybrid MPI + OpenMP Υλοποίηση Exclusive Scan	4
Εξήγηση Κώδικα.....	4
Παράδειγμα Εξόδου	5
c) Hybrid Parallel I/O για 3D Μητρώα	5
Εξήγηση Κώδικα.....	5
Σωστό Παράδειγμα Εκτέλεσης	6
Παράδειγμα Εκτέλεσης με Σφάλμα	6
d) Compressed Parallel I/O για 3D Μητρώα.....	7
Εξήγηση Κώδικα.....	7
Παράδειγμα Εκτέλεσης.....	7
Question 2: Parallel Parametric Search in Machine Learning	8
Επισκόπηση Δομής για το Ερώτημα 2	8
New Serial Code.....	8
Multiprocessing Pool Implementation.....	9
MPI Futures Implementation.....	11
MPI Master Worker Implementation	13
Question 3: OpenMP Tasking.....	16
a) Εξήγηση των επαναλήψεων του βρόγχου στα νήματα.....	16
b) OpenMP Tasks	17

Question 1: Hybrid Programming Model and Parallel I/O

Επισκόπηση Δομής για την Ερώτηση 1



Όπως φαίνεται στην εικόνα της δομής των αρχείων, η λύση χωρίζεται σε τέσσερις φακέλους (Subquestion A-D), για κάθε ένα από τα υποερωτήματα. Επιπλέον έχει δημιουργηθεί ο φάκελος headers, που περιέχει header files και υλοποιήσεις για συναρτήσεις που χρησιμοποιούνται σε πολλαπλά ερωτήματα, όπως η MPI_Exscan_omp.

Για το compilation και την εκτέλεση του κώδικα κάποιου MPI ερωτήματος, μπορείτε να μεταβείτε στον επιθυμητό φάκελο και να τρέξετε τις εντολές:

\$ make

\$ mpirun -np <num_processes> ./program

Για τα ερωτήματα με hybrid κώδικα MPI + OpenMP η εκτέλεση γίνεται ως εξής:

\$ mpirun -np <num_processes> ./program <num_threads>

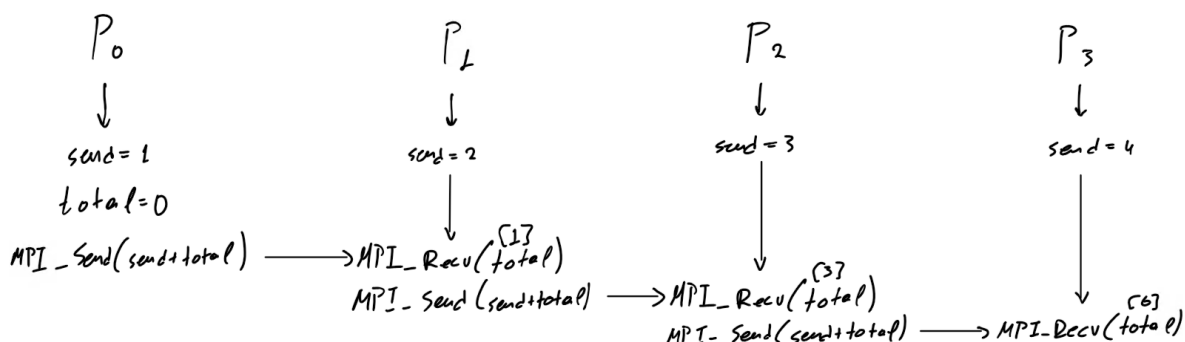
a) MPI Exclusive Scan Point-to-Point Υλοποίηση

Ο κώδικας του υποερωτήματος βρίσκεται στο αρχείο exscan_ptp.c. Η υλοποίηση της λειτουργίας Exscan με point-to-point επικοινωνία βρίσκεται στην συνάρτηση **Exscan_pt2pt(int *sendbuf, int *recvbuf, MPI_Comm comm)**.

Εξήγηση Κώδικα

Στην main, κάθε process διαθέτει ένα **send** και **total** buffer. Το send buffer περιέχει τον ακέραιο με τον οποίο το process θα συμβάλει στο μερικό άθροισμα του επόμενου. Ενώ το total buffer θα περιέχει το μερικό άθροισμα για το process, στο οποίο έχουν συμβάλει όλα τα προηγούμενα.

Παρακάτω απεικονίζεται η λογική της υλοποίησης για 4 processes διαγραμματικά.



Για το P0 θέτουμε το total σε 0, αφού δεν υπάρχει προηγούμενο process από το οποίο μπορεί να το λάβει. Στην συνέχεια στέλνει το άθροισμα του send και total buffer του στο επόμενο process. Όλα τα processes με rank > 0 κάνουν Recv αυτό το άθροισμα στο total buffer τους και στέλνουν το ίδιο άθροισμα στο επόμενο process, με εξαίρεση το τελευταίο.

Παράδειγμα Εξόδου

```
greg@thinkpad:~/Documents/GitHub/HPC-2024/Code/Question_1/Subquestion_A$ mpirun -np 4 ./exscan_ptp
Process 0: Sent = 1, Partial Reduction = 0
Process 1: Sent = 2, Partial Reduction = 1
Process 2: Sent = 3, Partial Reduction = 3
Process 3: Sent = 4, Partial Reduction = 6
```

b) Hybrid MPI + OpenMP Υλοποίηση Exclusive Scan

Ο κώδικας του υποερωτήματος βρίσκεται στο αρχείο `hybrid_exscan.c`. Η ζητούμενη υλοποίηση της συνάρτησης **MPI_Exscan_omp** βρίσκεται στο αρχείο `headers/mpi_exscan_omp.c`.

Για την εκτέλεση μπορείτε να τρέξετε.

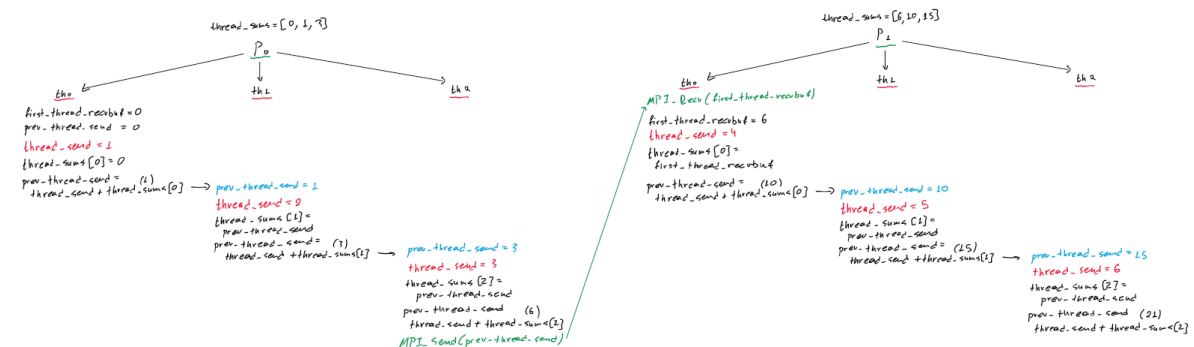
\$ make

\$ mpirun -np <num_processes> ./hybrid_exscan <num_threads>

Εξήγηση Κώδικα

Για να πραγματοποιηθεί η λειτουργία exclusive scan ανάμεσα στα threads ενός process πρέπει να καλέσουν την συνάρτηση `MPI_Exscan_omp` στην σειρά βάση του id τους. Αυτό γίνεται χρησιμοποιώντας ένα **omp for ordered** block. Φυσικά το πρώτο thread (εκτός από αυτό του πρώτου process) πρέπει να πραγματοποιήσει MPI point-to-point communication calls με το τελευταίο thread του προηγούμενου process για να λάβει το μερικό του άθροισμα από αυτό.

Παρακάτω φαίνεται η λογική της υλοποίησης για 2 processes με 3 threads διαγραμματικά.



Η τιμή του **thread_send** για το κάθε thread ορίζεται ως:

```
int thread_send = rank * omp_get_num_threads() + 1 + thread_id;
```

Έτσι, κάθε thread έχει διαδοχικούς ακέραιους αριθμούς από το 1 έως το `size * NUM_THREADS`.

Για λόγους απλότητας κυρίως στα επόμενα υποερωτήματα τα μερικά αθροίσματα του κάθε thread αποθηκεύονται σε έναν global πίνακα **thread_sums**.

Η inter-thread επικοινωνία για την μεταφορά του μερικού αθροίσματος στο επόμενο thread γίνεται με το global pointer **prev_thread_send**.

Παράδειγμα Εξόδου

```
greg@thinkpad:~/Documents/GitHub/HPC-2024/Code/Question_1/Subquestion_B$ mpirun -np 2 ./hybrid_exscan 3

Exscan results for process 0:
Process: 0 Thread 0: Sent: 1, Partial Reduction: 0
Process: 0 Thread 1: Sent: 2, Partial Reduction: 1
Process: 0 Thread 2: Sent: 3, Partial Reduction: 3

Exscan results for process 1:
Process: 1 Thread 0: Sent: 4, Partial Reduction: 6
Process: 1 Thread 1: Sent: 5, Partial Reduction: 10
Process: 1 Thread 2: Sent: 6, Partial Reduction: 15
```

c) Hybrid Parallel I/O για 3D Μητρώα

Ο κώδικας του υποερωτήματος βρίσκεται στο αρχείο `hybrid_matrix_io.c`. Ενώ οι βοηθητικές συναρτήσεις `create_random_matrices`, `print_matrices` και `validate_file` βρίσκονται στον φάκελο `headers`.

Για την εκτέλεση μπορείτε να τρέξετε.

\$ make

\$ mpirun -np <num_processes> ./hybrid_matrix_io <num_threads>

Εξήγηση Κώδικα

Κάθε thread αρχικοποιεί ένα 3D μητρώο μεγέθους $N \times N \times N$ με τυχαίες τιμές από το 0 έως το 99.

Παράδειγμα μητρώου:

```
Process 1:
Matrix for thread 0:
79 16 58
1 43 75
57 36 51

15 98 47
72 77 39
74 37 43

80 23 82
10 79 41
25 29 93
```

Έπειτα τα μητρώα, μετά από flattening σε μονοδιάστατους πίνακες, γράφονται παράλληλα σε ένα binary αρχείο (`matrices.bin`) μέσω της συνάρτησης `MPI_File_write_at(file, offset, flat_matrix, N * N * N, MPI_INT)`. Το offset εγγραφής του κάθε thread υπολογίζεται μέσω της συνάρτησης `MPI_Exscan_omp`, με το καθένα να στέλνει το μέγεθος του μητρώου του σε bytes ($N * N * N * \text{sizeof}(\text{int})$).

Τέλος πραγματοποιείται επιβεβαίωση της αριότητας των εγγεγραμμένων δεδομένων μέσω της συνάρτησης `validate_file("matrices.bin", matrices, thread_offsets, N, MPI_COMM_WORLD)`. Αυτή η συνάρτηση διαβάζει παράλληλα το μητρώο του κάθε thread και το συγκρίνει με το αρχικό.

Σωστό Παράδειγμα Εκτέλεσης

```
• greg@thinkpad:~/Documents/GitHub/HPC-2024/Code/Question_1/Subquestion_C$ mpirun -np 2 ./hybrid_matrix_io 4

Exscan results:
Process: 0 Thread 0: Sent: 108, Partial Reduction: 0
Process: 0 Thread 1: Sent: 108, Partial Reduction: 108
Process: 0 Thread 2: Sent: 108, Partial Reduction: 216
Process: 0 Thread 3: Sent: 108, Partial Reduction: 324
Process: 1 Thread 0: Sent: 108, Partial Reduction: 432
Process: 1 Thread 1: Sent: 108, Partial Reduction: 540
Process: 1 Thread 2: Sent: 108, Partial Reduction: 648
Process: 1 Thread 3: Sent: 108, Partial Reduction: 756

Validation Results for process 0:

Validation Results for process 1:
Process 0, Thread 0: Validation successful.
Process 1, Thread 3: Validation successful.
Process 1, Thread 1: Validation successful.
Process 0, Thread 3: Validation successful.
Process 0, Thread 1: Validation successful.
Process 1, Thread 2: Validation successful.
Process 1, Thread 0: Validation successful.
Process 0, Thread 2: Validation successful.
```

Παράδειγμα Εκτέλεσης με Σφάλμα

Για την επιβεβαίωση της σωστής λειτουργία της validation συνάρτησης, μπορούμε να αλλάξουμε τον πρώτο ακέραιο του αρχείου σε 100 (άκυρη τιμή). Ο πρώτος ακέραιος του αρχείου αντιστοιχεί στο μητρώο του πρώτου thread του πρώτου process.

```
// Change to first int in the file to 100
if(rank == 0){
    int invalid_entry = 100;
    MPI_File_write_at(file, 0, &invalid_entry, 1, MPI_INT, MPI_STATUS_IGNORE);
}

MPI_File_close(&file);
```

Έξοδος:

```
• greg@thinkpad:~/Documents/GitHub/HPC-2024/Code/Question_1/Subquestion_C$ mpirun -np 2 ./hybrid_matrix_io 4

Exscan results:
Process: 0 Thread 0: Sent: 108, Partial Reduction: 0
Process: 0 Thread 1: Sent: 108, Partial Reduction: 108
Process: 0 Thread 2: Sent: 108, Partial Reduction: 216
Process: 0 Thread 3: Sent: 108, Partial Reduction: 324
Process: 1 Thread 0: Sent: 108, Partial Reduction: 432
Process: 1 Thread 1: Sent: 108, Partial Reduction: 540
Process: 1 Thread 2: Sent: 108, Partial Reduction: 648
Process: 1 Thread 3: Sent: 108, Partial Reduction: 756

Validation Results for process 0:
Process 0, Thread 0: Validation failed.
Process 0, Thread 2: Validation successful.
Process 0, Thread 3: Validation successful.
Process 0, Thread 1: Validation successful.

Validation Results for process 1:
Process 1, Thread 0: Validation successful.
Process 1, Thread 2: Validation successful.
Process 1, Thread 1: Validation successful.
Process 1, Thread 3: Validation successful.
```

Φαίνεται πως η επιβεβαίωση απέτυχε για το μητρώο του πρώτου thread του πρώτου process.

d) Compressed Parallel I/O για 3D Μητρώα

Ο κώδικας του υποερωτήματος βρίσκεται στο αρχείο `compressed_matrix_io.c`. Ενώ οι βοηθητική συνάρτηση **`validate_compressed`** βρίσκεται στον φάκελο `headers`.

Για την εκτέλεση του κώδικα του υποερωτήματος απαιτείται η βιβλιοθήκη ZSTD.

```
$ sudo apt install libzstd-dev
```

Για την εκτέλεση μπορείτε να τρέξετε.

```
$ make
```

```
$ mpirun -np <num_processes> ./compressed_matrix_io <num_threads>
```

Εξήγηση Κώδικα

Η αρχικοποίηση των μητρώων γίνεται όπως στο προηγούμενο υποερώτημα.

Έπειτα οι πίνακες περνάνε από flattening και συμπιέζονται χρησιμοποιώντας την βιβλιοθήκη ZSTD. Τα συμπιεσμένα μητρώα και τα μεγέθη τους αποθηκεύονται στους πίνακες **`compressed_matrices`** και **`thread_matrix_sizes`**. Αξίζει να σημειωθεί, πώς μετά την συμπίεση, τα μητρώα δεν θα έχουν το ίδιο μέγεθος.

Το offset εγγραφής του κάθε thread υπολογίζεται μέσω της συνάρτησης `MPI_Exscan_omp`, με το καθένα να στέλνει το μέγεθος του συμπιεσμένου μητρώου και έπειτα πραγματοποιείται η παράλληλη εγγραφή τους στο αρχείο **`matrices_compressed.bin`**.

Τέλος, η εγκυρότητα των εγγεγραμμένων δεδομένων επιβεβαιώνεται με την συνάρτηση **`validate_compressed_file`**. Σε αυτή την συνάρτηση τα μητρώα αποσυμπιέζονται παράλληλα και συγκρίνονται με τα αρχικά.

Παράδειγμα Εκτέλεσης

```
greg@thinkpad:~/Documents/GitHub/HPC-2024/Code/Question_1/Subquestion_D$ mpirun -np 2 ./compressed_matrix_io 4

Exscan results:
Process: 0 Thread 0: Sent: 90, Partial Reduction: 0
Process: 0 Thread 1: Sent: 65, Partial Reduction: 90
Process: 0 Thread 2: Sent: 65, Partial Reduction: 155
Process: 0 Thread 3: Sent: 65, Partial Reduction: 220
Process: 1 Thread 0: Sent: 65, Partial Reduction: 285
Process: 1 Thread 1: Sent: 65, Partial Reduction: 350
Process: 1 Thread 2: Sent: 66, Partial Reduction: 415
Process: 1 Thread 3: Sent: 66, Partial Reduction: 481

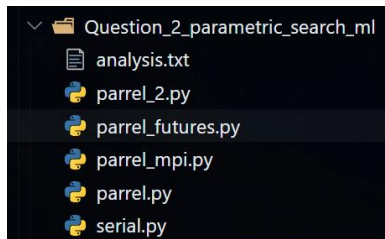
Validating compressed file.
Process 0, Thread 0: Validation successful.
Process 0, Thread 2: Validation successful.
Process 1, Thread 3: Validation successful.
Process 1, Thread 0: Validation successful.
Process 1, Thread 1: Validation successful.
Process 1, Thread 2: Validation successful.
Process 0, Thread 3: Validation successful.
Process 0, Thread 1: Validation successful.
```

Στα Exscan Results φαίνεται το γεγονός ότι τα μεγέθη των συμπιεσμένων μητρώων είναι διαφορετικά.

Αξίζει να αναφερθεί, πως για 8 μητρώα, το μέγεθος του `matrices_compressed.bin` είναι 547 bytes, ενώ του `matrices.bin` είναι 864 bytes. Έτσι επιβεβαιώνεται η αποτελεσματική συμπίεση των μητρώων

Question 2: Parallel Parametric Search in Machine Learning

Επισκόπηση Δομής για το Ερώτημα 2



Όπως φαίνεται στην εικόνα της δομής των αρχείων, η λύση χωρίζεται σε 5 αρχεία κώδικα python, 3 από τα οποία είναι για κάθε ένα από τα υποερωτήματα. Ένα για την βελτιωμένη σειριακή εκτέλεση του κώδικα το οποίο αναπτύχθηκε για τον σκοπό της συγκρίσεις της απόδοσης του με τις παράλληλες εκδοχές του κώδικα. Τέλος αναπτύξαμε ένα ακόμα version παραλληλίας για να δούμε εάν υπάρχει διαφορά μεταξύ του καθολικού data generation

και του data generation στο επίπεδο των διεργασιών.

Versions:

Python 3.12.7, scikit-learn version is 1.5.1.

System Resources for this subquestion 16 physical cores 32 threads (virtual cores).

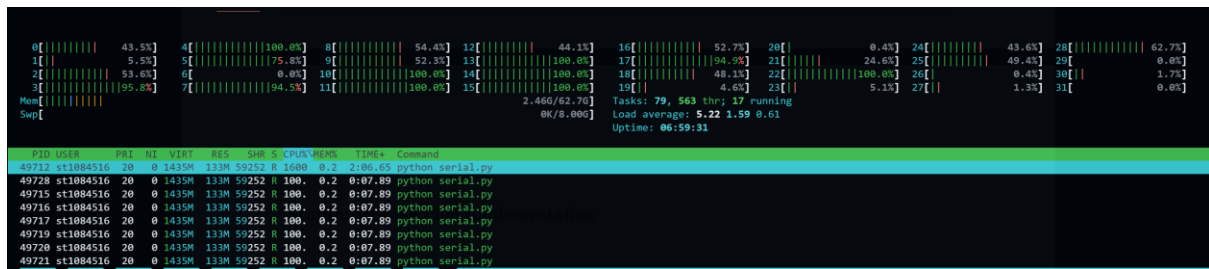
New Serial Code

Για την σύγκριση αποφασίσαμε να αλλάξουμε την δομή του σειριακού κώδικα που δόθηκε από την εκφώνηση `gs OG.py`, διότι απαιτείτε για την μετατροπή του στις παράλληλες εκδοχές με την χρήση `multiprocessing pool`, `mpi futures`, `master worker` λογική. Με απώτερο σκοπό να εξασφαλίσουμε τις πιο ορθές σύγκρισης μεταξύ της σειριακής εκδοχής του και των παράλληλων υλοποιήσεων του.

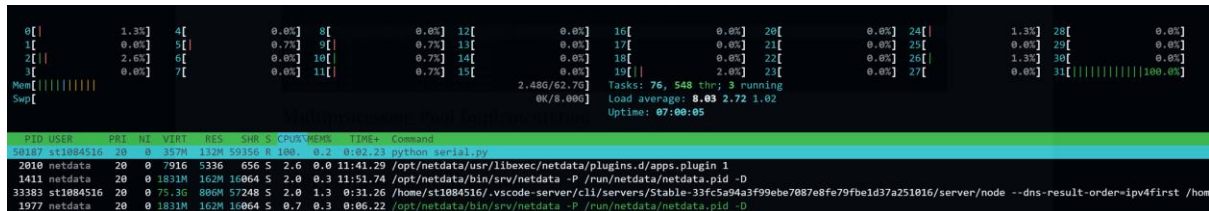
Η πρώτη διαφοροποίηση που κάναμε αφορά την δημιουργία δεδομένων στην original υλοποίηση τα δεδομένα δημιουργούνται μία φορά πριν από τον βρόχο εκπαίδευσης και χρησιμοποιούνται σταθερά για όλες τις δοκιμές. Στην δεύτερη εκδοχή τα δεδομένα δημιουργούνται μέσα στη συνάρτηση `train_and_evaluate` κάθε φορά που εκτελείται, με αποτέλεσμα να δημιουργείται νέο σύνολο δεδομένων για κάθε εκπαίδευση (κάναμε το assumption ότι αποφέρει μεγαλύτερη βελτιστοποίηση αφού ισοδύναμη με την παράλληλη φόρτιση του data set θα δούμε παρακάτω με συγκρίσεις). Στη συνέχεια αυξήσαμε τις διαστάσεις των νευρώνων από τρία επίπεδα (16 και 32) σε πέντε επίπεδα με μεγαλύτερες τιμές (128 και 256) και καθορίζεται το `max_iter=1000`, ενώ χρησιμοποιείται και ο αλγόριθμος 'adam' με συγκεκριμένο `random_state`. Αυτό έγινε για να καθιστούμε το αλγόριθμο πιο βάρη με απώτερο σκοπό να δούμε πιο ουσιώδεις διάφορες στην σύγκριση. Την πιο σημαντική αλλαγή αποτελεί η μετατροπή του βρόχου από την απευθείας επανάληψη για κάθε παραλλαγή του grid παραμέτρων (χωρίς ένδειξη index), στην επανάληψη που πραγματοποιεί εντός της συνάρτησης `train_and_evaluate` με εκτύπωση των παραμέτρων κάθε φορά που ξεκινά η εκπαίδευση.

Σε κάθε εκδοχή του κώδικα παρατηρήσαμε το εξής ενδιαφέρον πρόβλημα το οποίο οφείλετε τόσοι στην version της python όσο και την χρήση τελευταίας version της sklearn στην οποία οι developers έχουν αρχίσει να υλοποιούν κομμάτια πηγαίου κώδικα με version `omp` το οποίο οδηγεί σε default παράλληλη εκτέλεση του κώδικα πράγμα το οποίο οδηγεί με την επιπλέον υλοποίηση σε `multiprocessing Pool`, `mpi futures`, `master worker` σε διπλή παραλληλοποίηση στην ουσία τα `OpenMP threads` κάνουν παραγωγή extra διεργασιών και οδηγεί σε μεγάλη καθυστέρηση.

Την λύσης στο παραπάνω πρόβλημα δίνουμε με την χρήση της μεταβλητή περιβάλλοντος `OMP_NUM_THREADS` στο 1, για τον περιορισμό των `OpenMP` σε έναν νήμα.



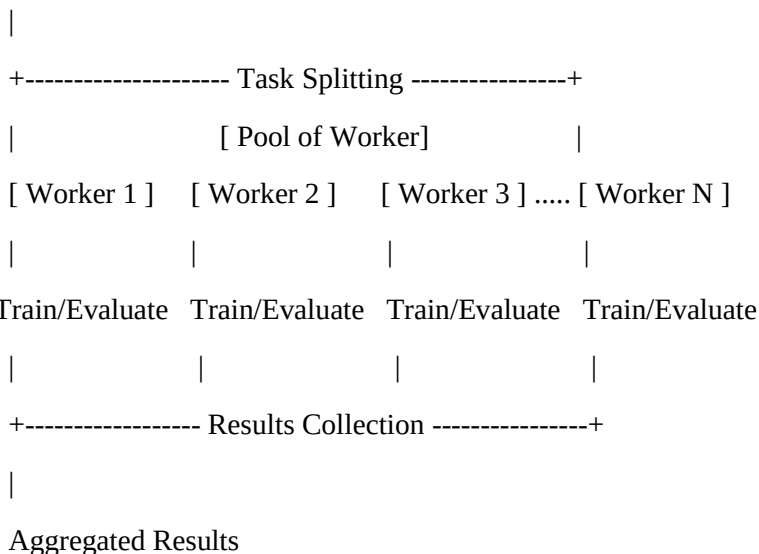
Η παραπάνω εικόνα δείχνει την εκτέλεση του serial χωρίς την μεταβλητή `OMP_NUM_THREADS = 1`



Η παραπάνω εικόνα δείχνει την εκτέλεση του serial με την μεταβλητή `OMP_NUM_THREADS = 1`

Multiprocessing Pool Implementation

Main Process



Η multiprocessing.Pool δημιουργεί μια δεξαμενή (Pool) διεργασιών (worker processes). Εν συνέχεια διασπά τις εργασίες σε τμήματα και τις αναθέτει στους εργαζόμενους. Κάθε εργαζόμενος λαμβάνει ένα τμήμα εργασιών, τις επεξεργάζεται ανεξάρτητα και επιστρέφει τα αποτελέσματα. Στην συγκεκριμένη περίπτωση. Κάθε διεργασία εκπαιδεύει και αξιολογεί το δικό της νευρωνικό δίκτυο ανεξάρτητα. Μόλις οι εργαζόμενοι ολοκληρώσουν τις εργασίες τους, επιστρέφουν τα αποτελέσματα (παραμέτρους και ακρίβεια) στον master.

Ο κώδικας αυτός έχει σχεδιαστεί για να δημιουργεί και να εκπαιδεύει ένα μοντέλο νευρωνικού δικτύου (MLPClassifier) χρησιμοποιώντας το scikit-learn. Αρχικά, ορίζεται η μεταβλητή περιβάλλοντος "OMP_NUM_THREADS" ώστε να περιοριστεί ο αριθμός των νημάτων σε ένα, προκειμένου να αποφευχθεί η υπερβολική παράλληλη εκτέλεση σε περιβάλλοντα που μπορεί να εκτελούνται ταυτόχρονα πολλές εργασίες. Στη συνέχεια, με τη βοήθεια της

συνάρτησης [make_classification](#), δημιουργείται ένα σύνολο δεδομένων με 1000 δείγματα. Το σύνολο αυτό διαιρείται σε δεδομένα εκπαίδευσης και δοκιμής.

Με την παράμετρο "ParameterGrid", ορίζεται ένα πλέγμα παραμέτρων που περιέχει διαφορετικές επιλογές για το μέγεθος των κρυφών στρωμάτων στο μοντέλο. Η συνάρτηση [train_and_evaluate](#) είναι υπεύθυνη για τη δημιουργία του dataset, την εκπαίδευση του μοντέλου και την αξιολόγηση της ακρίβειας μετρώντας σωστές προβλέψεις έναντι των πραγματικών τιμών.

Για τον παράλληλο υπολογισμό, χρησιμοποιείται η κλάση [Pool](#) από το module [multiprocessing](#). Με το [Pool.map](#), το πλέγμα των παραμέτρων διανέμεται στα διαθέσιμα worker processes ώστε να εκτελεστεί η συνάρτηση [train_and_evaluate](#) ανεξάρτητα για κάθε συνδυασμό παραμέτρων. Αυτό επιταχύνει την διαδικασία εκπαίδευσης και αξιολόγησης των διαφόρων πειραμάτων.

Η λειτουργία του Pool στην Python βασίζεται σε διάφορους worker processes που εκκινούν από τον κεντρικό διεργασία. Κάθε worker αναλαμβάνει να εκτελέσει την καθορισμένη συνάρτηση (σε αυτήν την περίπτωση, τη [train_and_evaluate](#)) για ένα σύνολο εισόδων. Μετά την ολοκλήρωση, τα αποτελέσματα συλλέγονται και επιστρέφονται στην κύρια διαδικασία. Αυτή η μέθοδος είναι ιδιαίτερα χρήσιμη για εργασίες υπολογιστικά απαιτητικές ή για εκτέλεση πολλών ανεξάρτητων εργασιών, καθώς επωφελείται από τους πολλούς πυρήνες του επεξεργαστή.

Τέλος, υπολογίζεται και εμφανίζεται ο συνολικός χρόνος που χρειάστηκε για την ολοκλήρωση των εργασιών, προσφέροντας έτσι μια ένδειξη απόδοσης της παραλληλοποίησης μέσω του multiprocessing.

Παρακάτω δίνεται μια πιο λεπτομερής εξήγηση για το πώς δουλεύει το multiprocessing.Pool σε χαμηλό επίπεδο:

Η υλοποίηση του Pool βασίζεται στη δημιουργία πολλών διεργασιών (processes) μέσω κλήσεων συστήματος όπως το fork (σε Unix/Linux σύστημα) ή το spawn (σε Windows). Κατά την εκκίνηση, ο κύριος διεργασία δημιουργεί έναν καθορισμένο αριθμό worker processes που έχουν ανεξάρτητη μνήμη και περιβάλλον εκτέλεσης. Αυτές οι διεργασίες είναι σχεδιασμένες να δέχονται εργασίες και να τις επεξεργάζονται παράλληλα.

Για τη διανομή των εργασιών, ο κύριος διεργασία αναλαμβάνει τη διαγραφή των εισερχόμενων δεδομένων σε μικρότερα κομμάτια (tasks) και τα στέλνει στους workers μέσω μηχανισμών επικοινωνίας μεταξύ διεργασιών (IPC), όπως είναι οι σωληνώσεις (pipes) ή οι ουρές (queues). Οι εργασίες και τα αποτελέσματα συχνά μεταφέρονται με χρήση serialization (η βιβλιοθήκη pickle της Python μετατρέπει αντικείμενα σε μια μορφή που μπορεί να σταλεί μεταξύ διεργασιών). Κάθε worker αναλαμβάνει την αποδοχή της εργασίας, την επεξεργασία της και την αποστολή πίσω του αποτελέσματος στον κύριο διεργασία.

Ο κύριος διεργασία παρακολουθεί την πρόοδο μέσω μηχανισμών συγχρονισμού, όπως events και locks, και συλλέγει τα αποτελέσματα από όλους τους workers. Όταν ολοκληρωθούν όλες οι εργασίες, το Pool αναλαμβάνει την καθαρή διακοπή των worker διεργασιών. Επίσης έχουμε κάνει cap τα process σε 8 Pool(processes=8)

Διότι παρατηρήσαμε ότι ο κώδικας τρέχει ποιο αποδοτικά από κάθε άλλη υλοποίηση αυτό οφείλετε είτε στην **Υπερφόρτωση και Εναλλαγή Πλαισίου (Context Switching)** αν και η CPU έχει 16 φυσικούς πυρήνες (32 νήματα), η δημιουργία πάρα πολλών διεργασιών (π.χ. 31) οδηγεί σε αυξημένη εναλλαγή πλαισίου και προγραμματιστικό κόστος.

Είτε στον ανταγωνισμό για Πόρους:

Πολλές διεργασίες ανταγωνίζονται για την Cache του CPU, το εύρος ζώνης της μνήμης και άλλους πόρους, γεγονός που μπορεί να επιβραδύνει κάθε μεμονωμένη διεργασία.

Φύση του Φορτίου Εργασίας:

Η εργασία σε κάθε διεργασία δεν είναι πολύ απαιτητική υπολογιστικά. Όταν το φορτίο είναι ελαφρύ, η υπερβολική παράλληλη επεξεργασία δημιουργεί επιπλέον διαχειριστικό κόστος σε σχέση με την απλή εργασία που πραγματοποιείται.

Συνοψίζοντας, ο αριθμός των οκτώ διεργασιών ισορροπεί παράλληλα την εκτέλεση και τα διαχειριστικά έξοδα, ενώ η χρήση 31 διεργασιών μπορεί να υπερφορτώσει το σύστημα με αυξημένο προγραμματιστικό και διαχειριστικό φόρτο, επηρεάζοντας την απόδοση

Τρόπος εκτέλεσης

`python3 serial.py`

`python3 parrel.py`

Αποτελέσματα εκτέλεσης

Total time taken (serial): 47.74 seconds

Total time taken: 9.87 seconds

`parrel_2.py`

Επίσης αναπτύξαμε και μια δεύτερη εκδοχή για testing καθαρά από ενδιαφέρον, στην οποία παράγουμε τα δεδομένα της εκπαίδευσης πριν το pool, το οποίο συνεπάγεται ότι δεν παράγονται μέσα σε κάθε worker και παρατηρήσουμε σχεδόν την ίδια απόδοση με το parrel 1.

Τρόπος εκτέλεσης

`python3 parrel_2.py`

Αποτελέσματα εκτέλεσης

Total time taken: 9.79 seconds

MPI Futures Implementation

Για την υλοποίηση του κώδικα με παράλληλη αναζήτηση υπερπαραμέτρων για έναν ταξινομητή MLP έγινε με τη χρήση MPI μέσω της βιβλιοθήκης mpi4py. Η βασική λογική της εφαρμογής παραμένει παρόμοια με την προηγούμενη υλοποίηση που χρησιμοποιούσε το multiprocessing Pool, αλλά η υλοποίηση με MPI επιτρέπει τη χρήση διαχειριστών εργασιών (futures) σε περιβάλλοντα υψηλότερης κλίμακας.

Διαφορές από την Υλοποίηση με Processes Pool

1. Μηχανισμός Παράλληλης Εκτέλεσης:

- Στην προηγούμενη υλοποίηση, η παράλληλη εκτέλεση επιτυγχάνεται με τη δημιουργία τοπικών διεργασιών μέσω της κλάσης Pool του multiprocessing.
- Στην τρέχουσα υλοποίηση, χρησιμοποιείται ο MPICommExecutor, ο οποίος αξιοποιεί το MPI (Message Passing Interface) για να διανείμει τις εργασίες μεταξύ των διαθέσιμων διεργασιών (ή κόμβων) που συμμετέχουν στον ίδιο communicator (MPI.COMM_WORLD). Αυτό επιτρέπει την εκτέλεση σε μεγαλύτερη κλίμακα, π.χ. σε cluster, και διευκολύνει τη διαχείριση του φόρτου εργασίας σε περιβάλλοντα κατανεμημένων συστημάτων.

2. Διαχείριση Εργασιών με Futures:

- Στην περίπτωση του multiprocessing Pool οι εργασίες παραδίδονται με τη μέθοδο `map`, ενώ οι επιστρεφόμενες τιμές συλλέγονται απευθείας.
- Στην υλοποίηση με MPI, ο `MPICommExecutor` παρέχει μια διεπαφή που ακολουθεί το μοντέλο των futures, όπου οι εργασίες υποβάλλονται (μέσω της `map` ή `submit` μεθόδου) και κάθε εργασία αναπαρίσταται από ένα future. Αυτά τα future αντικείμενα αποτυπώνουν την ασύγχρονη λειτουργία, επιτρέποντας στον χρήστη να παρακολουθεί την ολοκλήρωση και να συλλέγει τα αποτελέσματα μόλις αυτά είναι διαθέσιμα.

Εσωτερική Λειτουργία του `MPICommExecutor` και των Futures

Ο κώδικας ξεκινάει με την απόκτηση του βασικού communicator (`MPI.COMM_WORLD`). Αυτός ο communicator περιέχει όλους τους MPI διεργασίες που θα συμμετέχουν στην εκτέλεση. Ο `MPICommExecutor` δημιουργείται με βάση τον communicator που του έχει παρασχεθεί. Κατά την είσοδό του, ο executor διαμερίζει τις εργασίες μεταξύ των διαθέσιμων διεργασιών. Η μέθοδος `executor.map` λειτουργεί με τρόπο παρόμοιο με το `map` του παραδοσιακού `concurrent.futures`, υποβάλλοντας κάθε εργασία σε κάποια από τις απομακρυσμένες διεργασίες. Οι εργασίες εισάγονται στο εσωτερικό σύστημα διαχείρισης μηνυμάτων του MPI, το οποίο φροντίζει να μεταφέρει τις απαραίτητες πληροφορίες και δεδομένα. Κάθε εργασία που έχει υποβληθεί επιστρέφει ένα future, δηλαδή ένα αντικείμενο που αντιπροσωπεύει την ασύγχρονη εργασία. Το σύστημα MPI παρακολουθεί την ολοκλήρωση των υποβληθέντων εργασιών. Μόλις η διαδικασία ολοκληρωθεί και έχει επιστραφεί το αποτέλεσμα, το αντίστοιχο future ενημερώνεται ώστε να μπορέσει ο executor να συλλέξει τα αποτελέσματα.

Συγκέντρωση και Εμφάνιση Αποτελεσμάτων:

Μετά την ολοκλήρωση όλων των εργασιών, ο κώδικας μετατρέπει τα futures σε μια λίστα αποτελεσμάτων. Σημειώνεται ότι τα αποτελέσματα συλλέγονται και εμφανίζονται μόνο από την ρίζα του communicator (rank 0), διασφαλίζοντας ότι δεν υπάρχουν πολλαπλές εκτυπώσεις από τα διάφορα μέλη του συστήματος.

Συμπεράσματα

Η υλοποίηση με `MPICommExecutor` παρέχει μια πιο κλιμακούμενη προσέγγιση για την παράλληλη εκτέλεση, ιδανική για κατανεμημένα συστήματα και περιβάλλοντα HPC, ενώ παραμένει συμβατή με την φιλοσοφία χρήσης futures για ασύγχρονη διαχείριση εργασιών. Σε αντίθεση με την παραδοσιακή μέθοδο με το Pool, αυτή η προσέγγιση αξιοποιεί τις δυνατότητες του MPI για διευθέτηση και διαχείριση επικοινωνιών μεταξύ πολλών κόμβων, μειώνοντας έτσι πιθανές συμφορήσεις και βελτιώνοντας την αποδοτικότητα σε περιβάλλοντα με μεγάλο αριθμό διεργασιών.

Τρόπος εκτέλεσης

`mpirun -n 1 python parrel_futures.py`

Αποτελέσματα εκτέλεσης

Total time taken: 46.27 seconds

Τρόπος εκτέλεσης

`mpirun -n 2 python parrel_futures.py`

Αποτελέσματα εκτέλεσης

Total time taken: 25.12 seconds

Τρόπος εκτέλεσης

`mpirun -n 4 python parrel_futures.py`

Αποτελέσματα εκτέλεσης

Total time taken: 17.94 seconds

Τρόπος εκτέλεσης

`mpirun -n 8 python parrel_futures.py`

Αποτελέσματα εκτέλεσης

Total time taken: 9.93 seconds

Τρόπος εκτέλεσης

`mpirun -n 16 python parrel_futures.py`

Αποτελέσματα εκτέλεσης

Total time taken: 11.68 seconds

MPI Master Worker Implementation

Ρόλος Master-Worker:

Ο κώδικας διαχωρίζει τις διεργασίες σε master (rank 0) και workers (rank 1 έως size-1). Ο master κατανέμει τα καθήκοντα (παραμέτρους) μεταξύ όλων των διαθέσιμων διεργασιών, ενώ εκτελεί και αυτός μέρος των εργασιών του.

Διανομή Εργασιών:

Ο master δημιουργεί μια λίστα από παραμέτρους (pg) μέσω του ParameterGrid. Αργότερα, δημιουργεί έναν πίνακα με "tasks" για κάθε διεργασία, χρησιμοποιώντας round-robin κατανομή (i % size) ώστε να διανεμηθούν οι εργασίες ομοιόμορφα μεταξύ όλων των συμμετεχόντων διεργασιών.

Αλληλεπίδραση μέσω MPI:

Ο master στέλνει τα καθήκοντα (λίστες παραμέτρων) στους worker μέσω της εντολής `comm.send` (με tag 0).

Οι workers λαμβάνουν τις υποχρεώσεις τους, επεξεργάζονται κάθε μία παράμετρο καλώντας τη συνάρτηση `train_and_evaluate` και αποστέλλουν πίσω τα αποτελέσματα μέσω της εντολής `comm.send` (με tag 1).

Ο master λαμβάνει τα αποτελέσματα από όλους τους workers (χρησιμοποιώντας `comm.recv`) και τα συγχωνεύει με τα δικά του αποτελέσματα.

Εκτέλεση και Χρονική Μέτρηση:

Ο master μετρά τον συνολικό χρόνο εκτέλεσης ξεκινώντας πριν από την διανομή υφιστάμενων εργασιών και τερματίζοντας μετά τη συλλογή όλων των αποτελεσμάτων, για να παρουσιάσει την συνολική απόδοση.

Διαφορές από την υλοποίηση με MPICommExecutor

Μοντέλο Παραλληλισμού:

Στην περίπτωση του MPICommExecutor, ο χειρισμός των εργασιών έγινε με τη χρήση futures, όπου οι εργασίες υποβάλλονταν ασύγχρονα και ο executor συνεργαζόταν με το MPI για να διανείμει τα tasks.

Στην παρούσα υλοποίηση, ο προγραμματιστής χειρίζεται ρητά τη διανομή και τη συλλογή αποτελεσμάτων με τις βασικές εντολές [send](#) και [recv](#).

Το μοντέλο με MPICommExecutor απλοποιεί τη διαχείριση ασύγχρονων εργασιών χάρη στο ενσωματωμένο μοντέλο futures. Αντίθετα, η απευθείας χρήση των MPI primitives στο [parrel_mpi.py](#) απαιτεί περισσότερη διαχείριση από τον προγραμματιστή, δίνοντάς του όμως μεγαλύτερο έλεγχο για προσαρμοσμένη κατανομή εργασιών, σε περιβάλλοντα όπου ίσως χρειάζεται πιο λεπτή ρύθμιση των μηνυμάτων και των διαδικασιών.

Ο κάθε κόμβος καθορίζεται από ένα μοναδικό rank, ο οποίος επιτρέπει στον master να γνωρίζει ποιες ενέργειες πρέπει να γίνουν (αποστολή/λήψη μηνυμάτων). Αυτή η ιεραρχική δομή είναι κλασική στην προσέγγιση Master-Worker.

Χρήση MPI Tags και Διαχείριση Δεδομένων:

Τα tags στα μηνύματα (π.χ. tag 0 για αποστολή tasks και tag 1 για επιστροφή αποτελεσμάτων) επιτρέπουν την ορθή διαχείριση και συγχρονισμό των μηνυμάτων μεταξύ των διεργασιών. Ο master διατάσσει τα μηνύματα στους workers και στη συνέχεια περιμένει τα αποτελέσματα, εξασφαλίζοντας ότι όλη η εκτέλεση έχει συντονιστεί σωστά.

Τέλος, η υλοποίηση με MPI που παρουσιάζεται σε [parrel_mpi.py](#) παρέχει μια πιο «χαμηλού επιπέδου» προσέγγιση για την κατανεμημένη υπολογιστική παράλληλη αναζήτηση υπερπαραμέτρων, σε αντίθεση με το μοντέλο με MPICommExecutor που βασίζεται στους futures. Με τη ρητή χρήση των [send](#) και [recv](#):

Ο προγραμματιστής έχει απόλυτο έλεγχο στη διανομή των εργασιών.

Είναι δυνατή η υλοποίηση προσαρμοσμένων συστημάτων για πιο σύνθετες ή ειδικές απαιτήσεις επικοινωνίας και συγχρονισμού.

Παράλληλα, αυξάνει την πολυπλοκότητα της υλοποίησης και απαιτεί προσεκτικό σχεδιασμό για τη διαχείριση των μηνυμάτων.

Αυτή η προσέγγιση είναι κατάλληλη για περιβάλλοντα όπου ο έλεγχος της κατανομής εργασίας και η επικοινωνία μεταξύ κόμβων είναι κρίσιμα, π.χ., σε cluster υπολογιστών ή server racks, δίνοντας τη δυνατότητα για βελτιστοποίηση της επικοινωνίας και της διαχείρισης πόρων με βάση τις συγκεκριμένες απαιτήσεις της εφαρμογής.

Τρόπος εκτέλεσης

```
mpirun -n 1 python parrel_mpi.py
```

Αποτελέσματα εκτέλεσης

Total time taken: 45.72 seconds

Τρόπος εκτέλεσης

mpirun -n 2 python parrel_mpi.py

Αποτελέσματα εκτέλεσης

Total time taken: 25.27 seconds

Τρόπος εκτέλεσης

mpirun -n 4 python parrel_mpi.py

Αποτελέσματα εκτέλεσης

Total time taken: 14.31 seconds

Τρόπος εκτέλεσης

mpirun -n 8 python parrel_mpi.py

Αποτελέσματα εκτέλεσης

Total time taken: 10.70 seconds

Τρόπος εκτέλεσης

mpirun -n 16 python parrel_mpi.py

Αποτελέσματα εκτέλεσης

Total time taken: 11.46 seconds

Question 3: OpenMP Tasking

a) Εξήγηση των επαναλήψεων του βρόχου στα νήματα.

Λεπτομερής Επεξήγηση για το Πώς οι Επαναλήψεις Διαιρούνται στα Νήματα στον Παρεχόμενο Κώδικα OpenMP

Το παρεχόμενο από την εκφώνηση απόσπασμα κώδικα OpenMP έχει σχεδιαστεί για να αρχικοποιεί έναν πίνακα Α μεγέθους Ν χρησιμοποιώντας μια ασφαλή για νήματα και χρονοβόρα συνάρτηση work. Ο κώδικας χρησιμοποιεί το OpenMP για την παράλληλη εκτέλεση του βρόχου που γεμίζει τον πίνακα. Εδώ είναι το απόσπασμα κώδικα για αναφορά:

Η εντολή OpenMP που χρησιμοποιείται στον παρεχόμενο κώδικα είναι `#pragma omp parallel for schedule(dynamic, 2)`. Ας αναλύσουμε τι κάνει κάθε μέρος αυτής της εντολής:

`#pragma omp parallel for`:

Η `parallel` δημιουργεί μια ομάδα νημάτων, και το `for` διανέμει τις επαναλήψεις του βρόχου στα νήματα της ομάδας.

`schedule(dynamic, 2)`:

Η ρήτρα `schedule` καθορίζει πώς οι επαναλήψεις του βρόχου διανέμονται στα νήματα. Η δυναμική κατανομή σημαίνει ότι οι επαναλήψεις διαιρούνται σε τμήματα ενός καθορισμένου μεγέθους (στην προκειμένη περίπτωση, 2) και ανατίθενται στα νήματα δυναμικά. Κάθε νήμα ανατίθεται ένα τμήμα επαναλήψεων και όταν τελειώσει το τμήμα που του ανατέθηκε, ανατίθεται δυναμικά το επόμενο διαθέσιμο τμήμα. Αυτό συνεχίζεται μέχρι να ολοκληρωθούν όλες οι επαναλήψεις. Η παράμετρος στο `schedule(2)` καθορίζει το μέγεθος του τμήματος, που σημαίνει ότι κάθε νήμα θα ανατίθεται 2 επαναλήψεις τη φορά. Η OpenMP δημιουργεί μια ομάδα νημάτων (ας υποθέσουμε ότι υπάρχουν Τ νήματα διαθέσιμα).

Πιο συγκεκριμένα ο βρόχος έχει Ν επαναλήψεις, που πρέπει να διαιρεθούν μεταξύ των Τ νημάτων. Αρχικά, κάθε νήμα ανατίθεται ένα τμήμα 2 επαναλήψεων. Για παράδειγμα, το Νήμα 0 παίρνει τις επαναλήψεις 0 και 1, το Νήμα 1 παίρνει τις επαναλήψεις 2 και 3, και ούτω καθεξής. Μόλις ένα νήμα ολοκληρώσει το τμήμα που του ανατέθηκε, ζητά το επόμενο διαθέσιμο τμήμα. Αυτή η δυναμική ανάθεση εξασφαλίζει ότι τα νήματα διατηρούνται απασχολημένα και το φορτίο εργασίας είναι ισορροπημένο, ειδικά αν κάποιες επαναλήψεις παίρνουν περισσότερο χρόνο για να ολοκληρωθούν από άλλες.

Η δυναμική κατανομή είναι ιδιαίτερα χρήσιμη όταν το φορτίο εργασίας των επαναλήψεων είναι απρόβλεπτο ή ανισόροπο. Δεδομένου ότι η συνάρτηση `work` είναι χρονοβόρα, κάποιες επαναλήψεις μπορεί να πάρουν περισσότερο χρόνο για να ολοκληρωθούν από άλλες.

Με την δυναμική ανάθεση τμημάτων επαναλήψεων, το OpenMP εξασφαλίζει ότι τα νήματα που ολοκληρώνουν την διεργασία τους γρήγορα και έπειτα μπορούν να αναλάβουν επιπλέον επαναλήψεις, ισορροπώντας έτσι το φορτίο πιο αποτελεσματικά.

Παράδειγμα με Ψευδοκώδικα:

Ας εξετάσουμε ένα παράδειγμα με 4 νήματα ($T=4$) και 10 επαναλήψεις ($N=10$). Το μέγεθος του τμήματος είναι 2.

Αρχική ανάθεση:

Νήμα 0: Επαναλήψεις 0, 1

Νήμα 1: Επαναλήψεις 2, 3

Νήμα 2: Επαναλήψεις 4, 5

Νήμα 3: Επαναλήψεις 6, 7

Δυναμική Επανάθεση:

Μόλις το Νήμα 0 τελειώσει πρώτο, θα του ανατεθούν οι επαναλήψεις 8 και 9.

Αν το Νήμα 1 τελειώσει επόμενο, θα ελέγξει για διαθέσιμα τμήματα αλλά δεν θα βρει κανένα, καθώς όλες οι επαναλήψεις έχουν ολοκληρωθεί.

Αυτή η δυναμική προσέγγιση εξασφαλίζει ότι κανένα νήμα δεν μένει αδρανές ενώ υπάρχουν ακόμη επαναλήψεις που πρέπει να επεξεργαστούν.

Αυτή η προσέγγιση εξισορροπεί το φορτίο εργασίας και μειώνει τον χρόνο αδράνειας των νημάτων, καθιστώντας την κατάλληλη για σενάρια με απρόβλεπτο ή ανισόρροπο φορτίο εργασίας, όπως η χρονοβόρα συνάρτηση work στον δεδομένο κώδικα.

b) OpenMP Tasks

Στο πλαίσιο της εργασίας αυτής, μας ζητήθηκε να μετατρέψουμε τον υπάρχοντα δοθέντα κώδικα σε OpenMP παράλληλο αλγόριθμο με υλοποίηση που βασίζεται στο μοντέλο εργασιών (tasks).

Επιπλέον, αναλύσαμε και συγκρίναμε διαφορετικές προσεγγίσεις πολυνηματικής εκτέλεσης, ενώ επεκτείναμε σημαντικά το αρχικό πρόβλημα, δοκιμάζοντας διαφορετικές μεθόδους και βελτιστοποιήσεις

Παρακάτω παρατίθεται μια λεπτομερής αναφορά του κώδικα και των διαφορετικών υλοποιήσεων με OpenMP:

Η υλοποίηση σε σειριακή εκτέλεση (Serial) χρησιμεύει ως βασικό σημείο αναφοράς. Σε αυτήν, κάθε αριθμητική τιμή υπολογίζεται σειριακά, διατρέχοντας κάθε επανάληψη του βρόχου χωρίς παράλληλη επεξεργασία. Αυτή η υλοποίηση είναι απλή, αλλά αποδεικνύει το κόστος του υπολογισμού όταν δεν χρησιμοποιούνται πολλαπλοί πυρήνες.

Στην υλοποίηση με OpenMP threads, χρησιμοποιείται η οδηγία `#pragma omp parallel for schedule(dynamic, 2)`. Εδώ ο κύριος βρόχος καταμερίζεται δυναμικά μεταξύ των διαθέσιμων νημάτων. Κάθε νήμα αναλαμβάνει ένα κομμάτι των επαναλήψεων και εκτελεί την συνάρτηση `work(i)` για κάθε αντίστοιχο δείκτη. Ο δυναμικός προγραμματισμός επιτρέπει την πιο ισορροπημένη φόρτωση των νημάτων, κάτι που είναι κρίσιμο όταν ο χρόνος υπολογισμού ανά επανάληψη διαφέρει.

Η υλοποίηση με OpenMP Tasks δημιουργεί μία εργασία (task) για κάθε επανάληψη του βρόχου. Με την οδηγία `#pragma omp task firstprivate(i)` κάθε εργασία έχει το δικό της αντίγραφο του δείκτη, επιτρέποντας τη σωστή επεξεργασία χωρίς διασταυρούμενες μεταβλητές. Η χρήση της οδηγίας `#pragma omp taskwait` εξασφαλίζει ότι όλες οι εργασίες ολοκληρώνονται πριν συνεχιστεί η εκτέλεση. Η σημαντική διαφορά σε σχέση με τα απλά νήματα είναι ότι οι εργασίες μπορεί να εκτελεστούν ασύγχρονα και με μεγαλύτερη ευελιξία, αν και ενδέχεται να υπάρξει υψηλότερο κόστος δημιουργίας εργασιών όταν ο αριθμός τους είναι πολύ μεγάλος.

Η υλοποίηση με OpenMP Taskloop χρησιμοποιεί την οδηγία `#pragma omp taskloop grainsize(block_size)`, η οποία αυτοματοποιεί τη δημιουργία εργασιών για τις επαναλήψεις. Ο compiler αναλαμβάνει το διαχωρισμό του βρόχου σε μικρότερα κομμάτια (grains) βάσει της τιμής του block size. Αυτή η μέθοδος μειώνει την πολυπλοκότητα του κώδικα και παρέχει μια αποτελεσματική κατανομή των εργασιών, ωστόσο μπορεί σε ορισμένες περιπτώσεις να μην είναι τόσο γρήγορη όσο οι χειροκίνητα παραμετροποιημένες υλοποιήσεις με νήματα ή tasks, ειδικά αν το block size δεν επιλεγεί σωστά.

Η βελτιστοποιημένη υλοποίηση με `Tasks initialize_tasks_optimized`, όπου οι επαναλήψεις ομαδοποιούνται σε blocks, στοχεύει στη μείωση του overhead που συνδέεται με τη δημιουργία πολλών μικρών εργασιών. Με την οδηγία `#pragma omp task firstprivate(i, end) untied` δημιουργείται μία εργασία για κάθε block, η οποία επεξεργάζεται όλες τις επαναλήψεις μέσα σε αυτό το block. Η χρήση της οδηγίας `untied` επιτρέπει στο σύστημα εκτέλεσης (scheduler) να μην "δέσει" μια εργασία (task) σε ένα συγκεκριμένο νήμα από την αρχική της ανάθεση. Αυτό σημαίνει ότι, αν το νήμα που ξεκίνησε την εργασία καταστεί απασχολημένο ή μπλοκαριστεί για κάποιο λόγο, το σύστημα μπορεί να μεταφέρει την εργασία σε άλλο διαθέσιμο νήμα για να συνεχίσει την εκτέλεσή της. Σε περιβάλλοντα όπου οι εργασίες είναι αρκετά εκτενείς (δηλαδή δεν ολοκληρώνονται σε πολύ σύντομο χρονικό διάστημα) και ο χρόνος δημιουργίας της εργασίας (overhead) είναι κρίσιμος, αυτή η δυνατότητα μετακίνησης συμβάλλει στη βελτίωση της ισορροπίας φορτίου (load balancing). Συγκεκριμένα, ελαχιστοποιεί τους χρόνους αναμονής μεταξύ των νημάτων και διαχειρίζεται πιο αποτελεσματικά τους πόρους, οδηγώντας σε καλύτερη συνολική απόδοση του παραλληλισμού.

Η υλοποίηση με `reduction` χρησιμοποιεί την ενσωματωμένη ικανότητα της OpenMP να συνδυάζει (reduce) τιμές που παράγονται σε παράλληλες περιοχές χωρίς να απαιτείται ρητός συγχρονισμός για την ενημέρωση μιας κοινής μεταβλητής. Με την οδηγία `reduction(+:sum)` ο compiler και το runtime αναλαμβάνουν τη δημιουργία τοπικών αντιγράφων της μεταβλητής `sum` για κάθε νήμα ή task, και στο τέλος τα μερικά αποτελέσματα συγχωνεύονται με ασφαλή τρόπο.

Από την άλλη πλευρά, δεν μπορούσαμε να εφαρμόσουμε «απλές» `reduction` εξ ολοκλήρου στα tasks χωρίς να χρησιμοποιήσουμε τις συγκεκριμένες οδηγίες της OpenMP, επειδή:

1. **Ασύγχρονη Εκτέλεση των Tasks:** Οι tasks εκτελούνται ανεξάρτητα και η σειρά ολοκλήρωσής τους δεν είναι καθορισμένη. Χωρίς την οδηγία `reduction`, θα πρέπει να χειριστούμε ρητά τον συγχρονισμό για την πρόσβαση σε μια κοινή μεταβλητή, κάτι που μπορεί να οδηγήσει σε διαγωνισμό δεδομένων (data races) ή υπερβολικό κόστος συγχρονισμού.
2. **Ενιαία Ένωση Αποτελεσμάτων:** Η χρήση του `reduction(+:sum)` εξασφαλίζει ότι κάθε task επεξεργάζεται το μέρος του υπολογισμού και στη συνέχεια τα ενώνει με αποτελεσματικό τρόπο. Χωρίς αυτό, θα χρειαστεί να υλοποιήσουμε μια μηχανισμό συγχώνευσης (aggregation mechanism) οι οποίες πιθανώς απαιτούν μπλοκαρίσματα (locks) ή άλλες τεχνικές συγχρονισμού, αυξάνοντας το overhead και μειώνοντας την απόδοση.
3. **Διαχείριση Overhead:** Η ενσωματωμένη οδηγία `reduction` της OpenMP έχει σχεδιαστεί ώστε να ελαχιστοποιεί το επιπλέον κόστος, ενώ παρέχει ένα εύχρηστο API. Η γραφή μιας «απλής» λύσης `reduction` χωρίς να χρησιμοποιήσουμε τις οδηγίες που παρέχει η OpenMP είναι πολύπλοκη και σπάνια προσφέρει καλύτερη απόδοση από αυτήν που παρέχει το runtime.

Ο κύριος λόγος για τη χρήση της οδηγίας `reduction` είναι ότι εξασφαλίζει ασφαλή και αποδοτική συγχώνευση των αποτελεσμάτων που παράγονται σε παράλληλες tasks. Η «απλή» χρήση `reduction` χωρίς τις ειδικές οδηγίες της OpenMP δεν επαρκεί για τη σωστή συγχώνευση των τιμών, δεδομένων των απαιτήσεων για συγχρονισμό και ελαχιστοποίηση του overhead.

Για την ανάλυση των αποτελεσμάτων τρέξαμε τις παραπάνω υλοποιήσεις για μήκος array 1000, 10000, 100000 και αριθμό threads: 2, 4, 8, 16, 24, 28, 32 επίσης πέρα από την απλή χρονομέτρηση της εκτέλεσης κάθε διαφορετικής υλοποίησης υλοποιήσαμε μια συνάρτηση `max_difference` η οποία συγκρίνει τα αριθμητικά αποτελέσματα με τα αποτελέσματα τις σειριακής εκτέλεσης του κώδικα.

Η ανάλυση των αποτελεσμάτων δείχνει ότι στην περίπτωση του Array Size 1000, οι υλοποιήσεις με Threads και Tasks παρουσιάζουν τα καλύτερα αποτελέσματα σε όρους χρόνου εκτέλεσης, ειδικά όταν αυξάνεται ο αριθμός των νημάτων. Ο λόγος είναι ότι οι απευθείας παράλληλες υλοποιήσεις (με parallel for ή με task creation ανά επανάληψη) εκμεταλλεύονται πλήρως τους διαθέσιμους πυρήνες, ενώ οι implementations που βασίζονται σε taskloop και βελτιστοποιημένες tasks μπορεί να εισάγουν επιπλέον overhead λόγω παρεμβολών στο scheduling και τη διαχείριση των εργασιών. Επομένως, για το συγκεκριμένο πρόβλημα, όπου η εργασία ανά επανάληψη είναι αρκετά βαριά αλλά και σχετικά ανεξάρτητη, τις καλύτερες επιλογές φαίνεται να είναι είτε η προσέγγιση με threads είτε αυτή με tasks, δεδομένου ότι παρέχουν τον καλύτερο συνδυασμό απόδοσης και ευκολίας ισορροπίας φορτίου.

Συνοψίζοντας, ο κώδικας παρουσιάζει διαφορετικές τεχνικές παράλληλης επεξεργασίας με OpenMP, δίνοντας τη δυνατότητα δοκιμής κάθε προσέγγισης σε διαφορετικά μεγέθη προβλήματος και με διάφορους αριθμούς νημάτων.

Ακολουθούν μερικά ενδεικτικά αποτελέσματα:

===== Array Size: 100000 =====

Serial: ArraySize = 100000, Threads = 1, Time = 34.749497 sec

Threads: ArraySize = 100000, Threads = 4, Time = 8.928095 sec, MaxDiff = 0.000000e+00

Tasks: ArraySize = 100000, Threads = 4, Time = 8.950920 sec, MaxDiff = 0.000000e+00

Taskloop: ArraySize = 100000, Threads = 4, Time = 9.121523 sec, MaxDiff = 0.000000e+00

OptimizedTasks: ArraySize = 100000, Threads = 4, Time = 9.133554 sec, MaxDiff = 0.000000e+00

Reduction: ArraySize = 100000, Threads = 4, Time = 9.123278 sec, Sum = 224223557331.288361

Threads: ArraySize = 100000, Threads = 8, Time = 4.521591 sec, MaxDiff = 0.000000e+00

Tasks: ArraySize = 100000, Threads = 8, Time = 4.529524 sec, MaxDiff = 0.000000e+00

Taskloop: ArraySize = 100000, Threads = 8, Time = 4.710098 sec, MaxDiff = 0.000000e+00

OptimizedTasks: ArraySize = 100000, Threads = 8, Time = 4.659286 sec, MaxDiff = 0.000000e+00

Reduction: ArraySize = 100000, Threads = 8, Time = 4.651737 sec, Sum = 224223557331.287994

Threads: ArraySize = 100000, Threads = 16, Time = 2.360085 sec, MaxDiff = 0.000000e+00

Tasks: ArraySize = 100000, Threads = 16, Time = 2.362681 sec, MaxDiff = 0.000000e+00

Taskloop: ArraySize = 100000, Threads = 16, Time = 2.602132 sec, MaxDiff = 0.000000e+00

OptimizedTasks: ArraySize = 100000, Threads = 16, Time = 2.598168 sec, MaxDiff = 0.000000e+00

Reduction: ArraySize = 100000, Threads = 16, Time = 2.596357 sec, Sum = 224223557331.287689

Threads: ArraySize = 100000, Threads = 32, Time = 1.827815 sec, MaxDiff = 0.000000e+00

Tasks: ArraySize = 100000, Threads = 32, Time = 1.827426 sec, MaxDiff = 0.000000e+00

Taskloop: ArraySize = 100000, Threads = 32, Time = 2.076400 sec, MaxDiff = 0.000000e+00

OptimizedTasks: ArraySize = 100000, Threads = 32, Time = 2.070406 sec, MaxDiff = 0.000000e+00

Reduction: ArraySize = 100000, Threads = 32, Time = 2.062376 sec, Sum = 224223557331.287872

Επιπρόσθετα, για την ανάλυση των αποτελεσμάτων, αναπτύξαμε ένα script σε Python χρησιμοποιώντας τις βιβλιοθήκες pandas και matplotlib το οποίο παράγει τα εξής διαγράμματα

1. **Execution Time vs. Number of Threads** (Διαφορετικό διάγραμμα για κάθε μέγεθος πίνακα).
2. **Execution Time vs. Array Size** (Διαφορετικό διάγραμμα για κάθε αριθμό νημάτων).

